

System Design Preparation Roadmap

A step-by-step plan for interviews AND real-world understanding

PHASE 1: LEARN THE CORE BUILDING BLOCKS

Timeframe: Weeks 1-3

Before designing full systems, get comfortable with the individual pieces. Don't rush — spend real time understanding each term with simple explanations (short YouTube crash courses work well here).

- ***Load balancers***

What they do and why we need them to distribute traffic across servers.

- ***Caching (e.g. Redis)***

Why caching exists and how it dramatically speeds up systems.

- ***CDNs***

How content delivery networks make apps fast worldwide.

- ***SQL vs NoSQL databases***

The difference, and when to use each.

- ***API gateways***

What they do at the entry point of a system.

- ***Horizontal vs vertical scaling***

The two core ways to handle more load.

PHASE 2: UNDERSTAND KEY CONCEPTS (NOT JUST BUZZWORDS)

Timeframe: Alongside Phase 1

Learn WHY things exist, not just what they're called. Understanding the reasoning is what makes everything click instead of feeling like memorization.

- ***Why caching?***

To reduce load and improve speed for frequently accessed data.

- ***Why sharding?***

Because a single database machine can't hold or serve all the data at scale.

- ***Why message queues?***

To handle tasks asynchronously without blocking the user.

- ***Latency vs throughput***

Two different (and sometimes competing) measures of performance.

PHASE 3: STUDY SIMPLE, REAL EXAMPLES

Timeframe: Weeks 3-4

Look at how well-known simple systems are designed. These are small enough to fully understand end-to-end, and they use most of the concepts you just learned.

- ***URL shortener (like bit.ly)***

A great first case study — small enough to fully grasp, but touches databases, caching, and hashing.

- ***Rate limiter***

Teaches you about traffic control and protecting systems from abuse.

- ***Simple chat app***

Introduces real-time communication concepts.

PHASE 4: MOVE TO MEDIUM COMPLEXITY SYSTEMS

Timeframe: Weeks 5-6

Once comfortable with the basics, study slightly bigger designs. Watch or read how others break these down step-by-step so you see the thought process, not just the final diagram.

- ***Instagram-like feed***

Covers fan-out, ranking, and read-heavy system design.

- ***WhatsApp-like messaging***

Covers real-time delivery, offline handling, and message queues.

- ***Ride-sharing app (like Uber)***

Covers location tracking, matching algorithms, and geospatial data.

PHASE 5: PRACTICE EXPLAINING DESIGNS OUT LOUD

Timeframe: Ongoing

System design interviews are really about how you THINK, not memorized diagrams. Practice explaining your reasoning clearly, including trade-offs.

- ***Verbalize trade-offs***

Example: "I'd use SQL here for strong consistency in payments, but NoSQL for the activity feed since it's read-heavy and eventual consistency is fine."

- ***Practice with a partner***

Explain designs to a friend, mentor, or in a mock interview setting — or even to yourself out loud.

PHASE 6: DO TIMED MOCK PRACTICE

Timeframe: Weeks 7-8+

Once you know the basics, simulate real interview conditions to build genuine confidence.

- ***Pick a system, set a timer (30-45 min)***

Design it from scratch on paper or a whiteboard tool, using the standard framework below.

- ***Standard framework***

1) Clarify requirements 2) Estimate scale (users, data, traffic) 3) High-level design 4) Deep dive into 1-2 components 5) Discuss trade-offs and bottlenecks.

For Interviews Specifically

Learn and reuse the standard framework (clarify → estimate scale → high-level design → deep dive → trade-offs). Always narrate your reasoning out loud rather than jumping straight to a diagram. Do a handful of mock interviews before the real one.

For Real-World Use

On any project you work on, ask: what happens at 10x traffic? Where's the single point of failure? What's slow, and why? Read engineering blogs from companies like Uber, Netflix, and Airbnb — real systems, real trade-offs, no fluff.

Reminder: You don't need to be an expert in distributed systems to start. You just need the vocabulary and repeated practice applying it to simple examples. Confidence comes from repetition, not from knowing everything upfront.